OblivRec: Private Serving and Delivery

Mid-Term Technical Report, CS670

Mainak Sarkar (230619) Shrasti Dwivedi (230982) Aditya Anand (230065)
Shravan Agrawal (230984)

CS670: Cryptographic Techniques for Privacy Preservation
Indian Institute of Technology Kanpur
12 September 2026

Abstract

This report records the first implementation slice of OblivRec, a recommender that composes the three-party private training core of NUDGE with the private delivery layer of PIRSONA. The slice covers private serving and delivery against a model trained in the clear, which is stage S1 of the staging fixed in the requirements. We report a distributed point function written from the Boyle–Gilboa–Ishai construction, a DPF-based private information retrieval read layer over the MovieLens catalogue, a cleartext power-iteration factorisation used as the quality oracle, and an experiment quantifying what an observer learns from watching which items a user fetches. Private training itself is out of scope here and is scheduled for the following phase.

Contents

1 Scope of this slice

The full system has two halves. Private *training* learns a factorisation of a rating matrix that no server may see, and private *delivery* fetches the recommended item without revealing which item it was. The requirements document fixes the build order as S1, then S2, then S3: serving and delivery first, then training, then the composition. S1 is first because it is lower risk, it demonstrates something end to end early, and the distributed point function it needs is a prerequisite for the non-linear gates that private training will need later.

This report covers S1 only. The model is trained *in the clear* by the Python oracle of Section 2, and everything downstream of that model is private. Private training is the subject of the next phase and no claim is made about it here.

1.1 What is deliberately absent

Three components specified in the design draft are not built in this slice, and each was cut for a stated reason rather than for lack of time.

A distributed comparison function is not implemented. Its only consumers are the truncation and normalisation protocols, both of which belong to private training. The design draft asserts that a distributed point function and a distributed comparison function are the same primitive. They are not, and the reference implementation of Nudge keeps them in separate modules, which corroborates the distinction.

Oblivious top-k selection is not implemented. The user reconstructs the score vector and selects the top k locally, so no server learns the selection regardless of whether an oblivious selector exists. The apparatus would protect something that is already protected.

Network emulation is not used. The development machine has neither Docker nor `tc netem`, so every measurement in this report is taken on the `local` profile and is labelled as such. Wide-area numbers are deferred rather than estimated.

2 The cleartext oracle, and how many power iterations are needed

Every private component in this project is checked against a cleartext twin. The factorisation oracle is therefore the first thing built, before any cryptography, and it doubles as baseline B1, the speed-of-light reference against which recommendation quality is measured.

2.1 Construction

The oracle implements ApproxFactor as Nudge states it, rather than calling a singular value decomposition. This is deliberate. The private implementation in the next phase must reproduce this procedure step by step under secret sharing, so the reference has to match the *structure* of the protocol and not merely its output. Concretely, for each of d components a random vector is repeatedly multiplied by $\mathbf{U}^\top\mathbf{U}$, made orthogonal to the components already found, and normalised. Each converged component is then revealed in the clear and becomes a row of $\mathbf{B}$.

Revealing each row before computing the next is what keeps the orthogonalisation cheap, because it is then a Gram–Schmidt step against public vectors. It is also the reason the item embedding matrix is public to every server, which is the premise of the leakage analysis in this project.

A singular value decomposition is still computed, but as a *cross-check* rather than as the implementation. Power iteration on $\mathbf{U}^\top\mathbf{U}$ converges to the top-d right singular subspace, so the

largest principal angle between the two subspaces should approach zero. That check costs two lines and it caught a genuine question on the first day.

2.2 Quality is flat in the iteration count

At the default of $\ell = 10$ inner rounds the subspace angle is 9.66×10^{-1} radians, which is nowhere near zero. Sweeping ℓ resolves what that means.

Table 1: MovieLens-100K, split `u1`, $d = 16$. The subspace angle falls by five orders of magnitude while recommendation quality does not move. Raw data in `bench/results/oracle_ell_sweep.jsonl`.

ℓ	subspace angle (rad)	nDCG@20	Recall@20
10	9.659×10^{-1}	0.4536	0.4693
25	4.170×10^{-1}	0.4545	0.4677
50	1.834×10^{-1}	0.4525	0.4650
100	6.966×10^{-2}	0.4523	0.4662
200	3.604×10^{-3}	0.4526	0.4664
400	6.652×10^{-6}	0.4526	0.4664

Two conclusions follow, and they are separate.

The first is that the implementation is correct. The angle falls monotonically to 6.7×10^{-6} radians, so the procedure does converge to the right subspace. The large angle at $\ell = 10$ is slow convergence rather than a defect. The spectrum explains the rate: the ratio of the sixteenth to the seventeenth squared singular value is 1.032, a gap of three percent, and power iteration converges as a power of that ratio. The first component is easy, with a ratio of 6.44. The sixteenth is not.

This distinction was nearly lost. A failing cross-check invites the reaction of removing the cross-check, which would have converted a correct implementation into a silently wrong one.

The second conclusion is the useful one. Across a fortyfold increase in ℓ, nDCG@20 stays between 0.4523 and 0.4545, a spread of about half a percent with no monotone trend. Ranking does not require the singular vectors, only a subspace good enough that the ordering of scores is stable.

2.3 Why this matters for private training

Under two-out-of-three replicated sharing the matrix–vector products are non-interactive and therefore free. Truncation and normalisation are the entire communication cost of training, and they run once per inner iteration, so communication is linear in ℓ.

The measurement therefore says that private training can run at $\ell = 10$ and the fortyfold saving in communication costs nothing in recommendation quality. That is a design decision resting on evidence rather than on the default value in the design draft, and it also discharges an instruction the draft had already given and which had not been acted on, namely to measure the residual against the cleartext oracle and report iterations against quality.

The caveats are stated plainly. This is one dataset, one split, one seed and one embedding dimension. The flatness may not survive at MovieLens-1M or at $d = 32$, and it will be re-measured before the final report relies on it. The metric uses binary relevance at a rating of four or above, so these numbers are a within-project baseline and are not directly comparable to the figures NUDGE reports on Netflix data under its own convention.

References